🚀 Collections Framework Masterclass

Chapter 9 — LinkedHashSet (Complete Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐☆

`LinkedHashSet` is essentially a **HashSet + predictable insertion-order iteration**.

Since Chapter 8 already covered hashing, collisions, `equals()`, `hashCode()`, load factor, and resizing, we'll focus here on what is **new**.

---

1. What is LinkedHashSet?

`LinkedHashSet` is a `Set` implementation that:

- Does not allow duplicates
- Maintains **insertion order**
- Allows one `null`
- Provides average O(1) `add()`, `remove()`, and `contains()`
- Is not thread-safe

Package:

```
java.util.LinkedHashSet
```

Declaration:

```
public class LinkedHashSet<E>
    extends HashSet<E>
    implements Set<E>, Cloneable, Serializable
```

---

2. The Key Difference

The easiest way to remember it:

```
HashSet
    ↓
```

```
Uniqueness + Hashing

LinkedHashSet
    ↓
Uniqueness + Hashing + Insertion Order
```

Example:

```
Set<String> set = new LinkedHashSet<>();

set.add("Java");
set.add("Python");
set.add("Spring");
set.add("Java");
```

Iteration produces:

```
Java
Python
Spring
```

The duplicate `"Java"` is ignored, but its original position is retained.

---

3. How Does LinkedHashSet Preserve Order?

This is the main interview question.

`LinkedHashSet` uses a **LinkedHashMap** internally.

Conceptually:

```
LinkedHashSet
      ↓
LinkedHashMap
      ↓
HashMap
      +
Doubly linked list
```

The hash table provides efficient lookup.

The linked list maintains insertion order.

---

4. Internal Architecture

A normal HashSet essentially has:

```
HashSet
   ↓
HashMap
   ↓
Buckets
```

LinkedHashSet adds ordering:

```
LinkedHashSet
      ↓
LinkedHashMap
      ↓
Hash table + linked list
```

So each entry participates in two structures:

```
Hash table structure

AND

Doubly linked list
```

---

5. Simplified Internal Representation

Imagine:

```
set.add("A");
set.add("B");
set.add("C");
```

Hash table:

```
Bucket
  ↓
[A]

Bucket
  ↓
[B]

Bucket
  ↓
[C]
```

Linked-list order:

```
A ⇄ B ⇄ C
```

The hash table answers:

> "Where is this element?"

The linked list answers:

> "What is the iteration order?"

---

6. `before` and `after` References

This is an important internal detail.

LinkedHashMap entries maintain links conceptually like:

```
Entry

+--------+------+--------+
| before | key  | after  |
+--------+------+--------+
```

Example:

```
null ← A ⇄ B ⇄ C → null
       ↑         ↑
      head      tail
```

Therefore, when iterating, Java can follow the linked-list chain instead of scanning the hash table.

---

7. Why Can't HashSet Simply Preserve Order?

HashSet's iteration order is determined by its hash-table structure.

If elements are placed into different buckets:

```
A → Bucket 5
B → Bucket 1
C → Bucket 7
```

there is no separate structure remembering:

```
A → B → C
```

LinkedHashSet adds that structure.

---

8. What Happens During `add()`?

Consider:

```
set.add("Java");
```

Conceptually:

```
add("Java")
     ↓
Hashing
     ↓
Find bucket
     ↓
Duplicate?
```

```
   ↙       ↘
 Yes       No
  ↓         ↓
Reject    Insert
             ↓
       Link into list
```

So a successful insertion:

1. Places the entry in the hash table.
2. Links the entry into the insertion-order chain.

---

9. What Happens with a Duplicate?

```
set.add("Java");
set.add("Java");
```

First:

```
Java → inserted
```

Second:

```
Java → already exists
       ↓
     reject
```

No second linked-list node is created.

Therefore:

```
Java ⇄ Python ⇄ Spring
```

not:

```
Java ⇄ Python ⇄ Spring ⇄ Java
```

---

10. Does `remove()` Preserve the Remaining Order?

Yes.

Suppose:

```
A ⇄ B ⇄ C ⇄ D
```

Remove `B`.

The links are adjusted:

```
A ⇄ C ⇄ D
```

Iteration now produces:

```
A
C
D
```

The relative order of remaining elements is preserved.

---

11. Re-Adding an Existing Element

Important interview trap.

```
Set<String> set =
    new LinkedHashSet<>();

set.add("A");
set.add("B");
set.add("C");

set.add("A");
```

Result:

```
[A, B, C]
```

It does **not** move `A` to the end.

Why?

Because the second `add("A")` is a duplicate and does not create a new insertion.

---

12. LinkedHashSet vs HashSet

Feature	HashSet	LinkedHashSet
Duplicates	No	No
Ordering	No guarantee	Insertion order
Average `add()`	O(1)	O(1)
Average `contains()`	O(1)	O(1)
Average `remove()`	O(1)	O(1)
Memory	Lower	Higher
Internal backing	HashMap	LinkedHashMap

The key trade-off:

```
LinkedHashSet

More memory
     +
Slightly more bookkeeping
     ↓
Predictable iteration order
```

---

13. LinkedHashSet vs TreeSet

Don't confuse these.

LinkedHashSet

```
Insertion order
```

TreeSet

```
Sorted order
```

Example input:

```
40, 10, 30, 20
```

LinkedHashSet:

```
40, 10, 30, 20
```

TreeSet:

```
10, 20, 30, 40
```

---

14. LinkedHashSet vs TreeSet

Feature	LinkedHashSet	TreeSet
Ordering	Insertion order	Sorted order
Typical complexity	O(1) average	O(log n)
Internal structure	Hash table + linked list	Red-Black Tree
Duplicates	No	No
Null	One null allowed	Generally no null
Range operations	No	Yes

Choose based on the required ordering semantics.

---

15. Null Handling

LinkedHashSet allows one `null`.

```
Set<String> set =
    new LinkedHashSet<>();

set.add(null);
set.add(null);
set.add("Java");
```

Iteration:

```
null
Java
```

Only one null exists because duplicates aren't allowed.

---

16. Performance

For normal operations:

```
add()      → O(1) average
contains() → O(1) average
remove()   → O(1) average
```

Iteration is:

```
O(n)
```

One useful distinction:

> LinkedHashSet iteration is based on the linked insertion-order chain, whereas HashSet iteration traverses the hash-table structure.

---

17. Why Can LinkedHashSet Iteration Be Better Than HashSet?

Suppose:

```
Size = 100
Capacity = 1,000,000
```

HashSet iteration can involve scanning the underlying table structure.

LinkedHashSet maintains a linked chain containing only actual entries.

So LinkedHashSet iteration is proportional to the number of elements rather than the hash-table capacity.

This is an important performance detail.

---

18. Memory Overhead

LinkedHashSet requires additional links to maintain order.

Conceptually, each entry needs:

```
Hash information
Key
Value/dummy value
Next link
Previous link
```

Therefore:

```
LinkedHashSet
    >
HashSet
```

in memory usage.

The exact footprint depends on JVM implementation and object layout.

---

19. When Should You Use LinkedHashSet?

Use it when you need:

```
Unique elements
      +
Original insertion order
```

Examples:

Remove duplicates while preserving order

```
List<String> names =
    Arrays.asList(
        "A", "B", "A", "C", "B"
    );

Set<String> unique =
    new LinkedHashSet<>(names);
```

Result:

```
[A, B, C]
```

API Response

Suppose an API must return unique values while preserving the order in which they were encountered.

```
Set<String> result =
    new LinkedHashSet<>();
```

Ordered permissions

```
Set<String> permissions =
    new LinkedHashSet<>();
```

Useful when permissions must be unique but displayed in configuration order.

20. When Should You NOT Use LinkedHashSet?

Don't need ordering

Use:

```
HashSet
```

Need sorted order

Use:

```
TreeSet
```

Need duplicates

Use:

```
List
```

Need index-based access

Use:

```
ArrayList
```

---

21. Is LinkedHashSet Thread-Safe?

No.

Just like HashSet, LinkedHashSet is not synchronized.

For synchronized access:

```
Set<String> set =
    Collections.synchronizedSet(
```

```
        new LinkedHashSet<>()
    );
```

For concurrent workloads, choose a collection designed for the required concurrency semantics rather than automatically wrapping everything.

---

22. Fail-Fast Behavior

Its iterator is generally fail-fast.

Example:

```
for (String value : set) {

    if (value.equals("B")) {
        set.remove(value);
    }
}
```

This can result in:

```
ConcurrentModificationException
```

Use the iterator's own `remove()` when removing during iteration:

```
Iterator<String> iterator =
    set.iterator();

while (iterator.hasNext()) {

    String value = iterator.next();

    if (value.equals("B")) {
        iterator.remove();
    }
}
```

As discussed earlier, fail-fast behavior is **best effort**, not a synchronization mechanism.

---

23. LinkedHashSet Does NOT Mean Sorted

This is one of the easiest interview traps.

Input:

```
50
10
30
20
```

LinkedHashSet:

```
50
10
30
20
```

It preserves **insertion order**.

It does NOT sort.

If you need:

```
10
20
30
50
```

use `TreeSet`.

---

24. ⭐ Interview Scenario

Question

You receive millions of product IDs from a stream.

You need to:

1. Remove duplicates

2. Preserve the first-seen order
3. Iterate over the result later

Which collection?

Answer

```
LinkedHashSet<Long>
```

Because it provides:

```
Uniqueness
+
Insertion-order iteration
```

---

25. ⭐ Interview Scenario

Question

You have:

```
A
B
C
A
D
B
```

You need:

```
A
B
C
D
```

Use:

```
Set<String> unique =
    new LinkedHashSet<>(values);
```

Why not HashSet?

Because HashSet does not guarantee the original order.

---

26. ⭐ Interview Scenario

Question

Why would you choose LinkedHashSet instead of HashSet if both have O(1) average lookup?

Because the requirement is not just lookup.

You also need:

```
Predictable insertion-order iteration
```

The additional linked-list bookkeeping provides that guarantee.

---

27. Frequently Asked Interview Questions

Q1. How does LinkedHashSet maintain insertion order?

Through its underlying `LinkedHashMap`, which maintains a doubly linked list connecting entries in insertion order.

Q2. Does LinkedHashSet use HashMap internally?

More precisely, `LinkedHashSet` is implemented using `LinkedHashMap`-based infrastructure.

Q3. Does LinkedHashSet allow duplicates?

No.

Q4. Does LinkedHashSet allow null?

Yes, one `null`.

Q5. Does LinkedHashSet sort elements?

No.

It maintains insertion order.

Q6. What is the difference between HashSet and LinkedHashSet?

The primary difference is ordering:

```
HashSet
→ no ordering guarantee

LinkedHashSet
→ insertion-order iteration
```

LinkedHashSet also has additional memory/bookkeeping overhead.

Q7. What happens if an existing element is added again?

The insertion is rejected and the element retains its original position.

Q8. Is LinkedHashSet thread-safe?

No.

Q9. What is the average complexity of `contains()`?

O(1).

Q10. Why does LinkedHashSet consume more memory than HashSet?

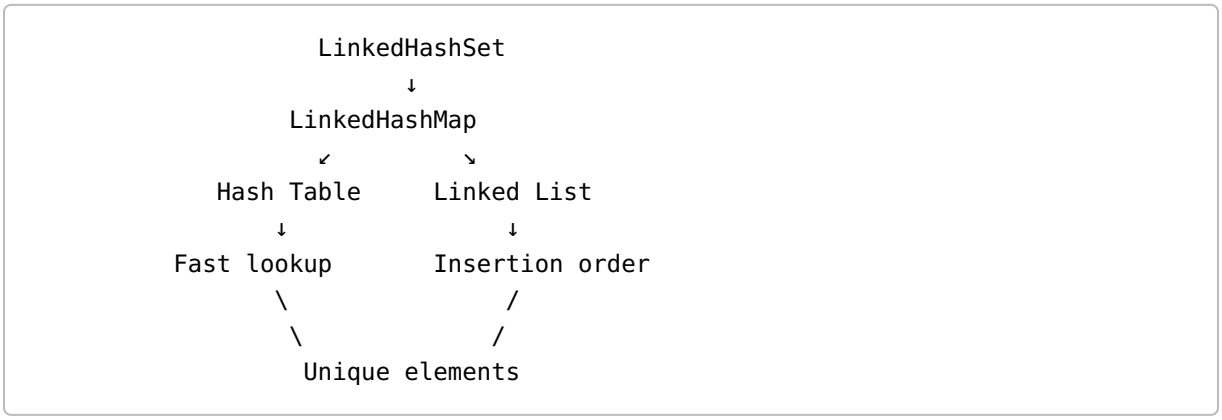
Because entries maintain additional links needed to preserve insertion order.

---

28. Decision Table

Requirement	Choose
Unique + fast lookup	`HashSet`
Unique + insertion order	`LinkedHashSet`
Unique + sorted order	`TreeSet`
Duplicates + index access	`ArrayList`
Queue/Deque	`ArrayDeque`

🧠 Final Mental Model



That's the key idea.

HashSet answers "Is this element present?"

LinkedHashSet answers "Is this element present, and in what insertion order should I iterate?"

🔥 Chapter Summary

Concept	LinkedHashSet
Interface	Set
Duplicates	Not allowed
Null	One allowed

Concept	LinkedHashSet
Ordering	Insertion order
Sorting	No
Average `add()`	O(1)
Average `contains()`	O(1)
Average `remove()`	O(1)
Iteration	O(n)
Thread-safe	No
Internal infrastructure	LinkedHashMap
Extra structure	Doubly linked ordering chain
Memory	Higher than HashSet
Best use	Unique + insertion-order iteration

🎯 Interview Readiness Checklist

You should now be able to answer:

- [ ] How does LinkedHashSet preserve insertion order?
- [ ] HashSet vs LinkedHashSet?
- [ ] LinkedHashSet vs TreeSet?
- [ ] Why does LinkedHashSet use more memory?
- [ ] What happens when an existing element is added again?
- [ ] Does LinkedHashSet sort elements?
- [ ] Can LinkedHashSet contain null?
- [ ] Is LinkedHashSet thread-safe?
- [ ] Why is lookup still O(1) average?
- [ ] How would you remove duplicates while preserving order?

🚀 Next Chapter

Chapter 10 — TreeSet (Complete Deep Dive)

We'll focus on what's new:

- NavigableSet and SortedSet

• Red-Black Tree internals
• Natural ordering
• `Comparable` vs `Comparator`
• How TreeSet detects duplicates
• Why `compareTo() == 0` means "duplicate"
• `first()`, `last()`, `higher()`, `lower()`, `ceiling()`, `floor()`
• Range views: `subSet()`, `headSet()`, `tailSet()`
• O(log n) performance
• Null handling
• Mutable elements and ordering problems
• 20+ tricky interview questions